МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по выполнению индивидуального задания

по дисциплине «Технология разработки программного обеспечения»

по теме: «Создание подсистемы для работы с партнерами компании»

Работу выполнил

студент группы ИПо-42

У.Е.Посохина

«16» марта 2026 г.

Пермь, 2026 г.

Оглавление

Введение 3
Глава 1. Анализ предметной области 4
1.1 Описание предметной области 4
1.2 Функциональные характеристики системы 5
Глава 2. Создание базы данных 7
2.1 Логическая модель данных 7
2.2 Практическая реализация структуры данных 7
Глава 3. Создание программных библиотек 12
Глава 4. Тестирование библиотеки 20
4.1 Проверка работоспособности компонентов с помощью модульных тестов 20
4.2 Результаты тестирования 22
Глава 5. Создание клиентского приложения 24
5.1 Разработка пользовательского интерфейса 24
5.2 Построение окон приложения 25
Заключение 31
Приложения 32

Введение

Данный отчет содержит описание этапов анализа предметной области организации, для которой создавалась система, разработки логической модели данных, реализации программного продукта и его проверки. Готовое приложение предоставляет пользователю возможность работы с информацией через оконный графический интерфейс.

В ходе разработки использовались следующие инструменты:

- реляционная СУБД PostgreSQL для организации хранения данных;
- среда разработки Visual Studio 2022;
- платформа WPF для построения интерфейса пользователя.

Функциональная часть приложения построена на основе библиотеки, поддерживающей четыре базовых действия с данными: ввод новой информации, просмотр существующих записей, корректировка данных и удаление записей.

Для организации обмена данными между приложением и базой использованы дополнительные библиотеки:

- Microsoft.EntityFrameworkCore — обеспечивает отображение таблиц базы данных на объекты языка C#;
- Npgsql.EntityFrameWorkCore.PostgreSQL — предоставляет поддержку PostgreSQL для Entity Framework;
- Microsoft.EntityFrameworkCore.Tools — содержит вспомогательные функции для синхронизации кода и структуры базы данных.

Основной целью работы являлось создание программного средства для управления данными, организованными по принципу связи «один ко многим». Интерфейс пользователя реализован средствами WPF, а взаимодействие с информацией в базе данных — через механизмы Entity Framework Core.

Глава 1. Анализ предметной области

1.1 Описание предметной области

Анализ предметной области и определение требований к системе

До начала разработки был выполнен анализ информации, с которой предстоит взаимодействовать пользователю, и установлен перечень данных, подлежащих изменению. Полное описание предметной области представлено в приложении 1.

В соответствии с поставленной задачей необходимо было реализовать программный модуль для автоматизации учета контрагентов предприятия, обеспечивающий следующие функции:

- просмотр полного перечня партнеров;
- добавление новых записей и изменение существующих данных о контрагентах;
- отображение информации о продажах, совершенных каждым партнером.

При формировании сведений о продажах требуется выводить:

- наименование товара;
- количество реализованной продукции;
- дату совершения сделки.

В карточке партнера должны присутствовать поля для ввода и отображения следующих данных:

- полное наименование организации;
- категория контрагента;
- числовое значение рейтинга;
- почтовый адрес;
- фамилия, имя, отчество руководителя;
- контактный телефон;
- адрес электронной почты.

Дополнительно техническим заданием предусмотрена возможность выполнения операций добавления, изменения и удаления не только для

записей о партнерах, но и для связанных с ними объектов: категорий партнеров, справочника продукции и данных о реализованных товарах.

1.2 Функциональные характеристики системы

На основе анализа требований была сформирована структура базы данных, включающая четыре основные таблицы:

- справочник товаров;
- перечень категорий партнеров;
- регистр контрагентов;
- журнал продаж, совершенных партнерами.

Таблица товаров содержит два поля: уникальный артикул, служащий для идентификации продукции, и наименование. Иные характеристики (цена, изображение, описание) в рамках поставленной задачи не требовались и при разработке не учитывались.

Таблица категорий партнеров включает уникальное наименование категории без дополнительных атрибутов.

Таблица партнеров, помимо перечисленных выше реквизитов, содержит ссылку на категорию, к которой относится контрагент.

Таблица продаж фиксирует информацию о реализованной продукции: ссылку на товар, ссылку на партнера-продавца, количество проданных единиц и дату совершения операции.

Проведенный анализ позволил определить состав сущностей и их атрибутов, необходимых для последующего проектирования базы данных разрабатываемой системы.

Дополнительные функциональные требования:

- автоматический пересчет скидки партнера при любых изменениях данных о продажах;
- зависимость скидки от общего количества проданной продукции по следующей шкале:
    1. до 10 000 единиц – скидка 0%;
    2. от 10 001 до 50 000 единиц – скидка 5%;

3. от 50 001 до 300 000 единиц – скидка 10%;
4. более 300 000 единиц – скидка 15%.

Требования к пользовательскому интерфейсу:

- разделение главного окна на две области: слева – список партнеров, справа – история продаж;
- наличие главного меню для доступа ко всем функциям;
- наличие контекстного меню (по правой кнопке мыши) для быстрого выполнения операций;
- наличие диалоговых окон для ввода и редактирования данных;
- информативные сообщения об ошибках с указанием причин и способов их исправления;
- подтверждение критических действий (удаление, выход из программы).

Глава 2. Создание базы данных

2.1 Логическая модель данных

В соответствии с техническим заданием, программный продукт должен функционировать с заранее подготовленной базой данных и использовать определенную учетную запись для подключения. Поэтому перед началом написания программного кода выполняются подготовительные работы по созданию инфраструктуры хранения информации.

В рамках этих работ создается учетная запись (роль), наделенная необходимыми правами доступа, формируется табличное пространство для размещения информации, инициализируется сама база данных, и внутри нее организуется отдельная схема, владельцем которой становится созданная ранее роль.

Взаимосвязи между объектами данных выглядят следующим образом:

- «Категория контрагента» и «Контрагент» — отношение типа «один ко многим»: одна категория может быть присвоена нескольким контрагентам;
- «Контрагент» и «Реализация» — отношение типа «один ко многим»: один контрагент может осуществлять множество продаж;
- «Товар» и «Реализация» — отношение типа «один ко многим»: один товар может фигурировать во многих операциях продажи.

Все перечисленные связи поддерживают ссылочную целостность данных: при удалении записи о контрагенте все связанные с ним операции продаж также удаляются; при попытке удалить категорию или товар, имеющие связанные записи, выполнение операции блокируется.

2.2 Практическая реализация структуры данных

Перечисленные операции выполняются посредством команд, вводимых в терминале PostgreSQL. Для направления запросов задействуется стандартный консольный клиент psql. Полный список использованных команд с необходимыми пояснениями представлен в листинге 2.1.

Листинг 2.1 – Набор команд для инициализации и управления базой данных

```
\! chcp 1251

-- Создание роли
CREATE ROLE app3 WITH LOGIN PASSWORD '123456789';

-- Создание базы данных
CREATE DATABASE posohina
    WITH OWNER = app3
    ENCODING = 'UTF8'
    LC_COLLATE = 'ru_RU.UTF-8'
    LC_CTYPE = 'ru_RU.UTF-8'
    TEMPLATE = template0;

-- Подключение к базе данных
\c posohina app3;

-- Создание схемы
CREATE SCHEMA business AUTHORIZATION app3;
```

Для создания учетной записи, обеспечивающей доступ к базе данных, были заданы следующие характеристики:

- идентификатор роли: «app3»;
- активированы параметры «LOGIN» (разрешение на подключение) и «PASSWORD» (установка пароля);
- значение пароля для входа: «123456789».

После инициализации базы данных осуществляется формирование таблиц в соответствии с разработанной логической структурой. Команды, используемые для создания таблиц, приведены в листинге 2.2.

Листинг 2.2 – Создание таблиц базы данных

```
-- Создание таблицы партнеров
CREATE TABLE partners_posohina (
    id SERIAL PRIMARY KEY,
    type_id INTEGER NOT NULL REFERENCES partner_types_posohina(id) ON DELETE RESTRICT,
    name VARCHAR(200) NOT NULL,
    director_fullname VARCHAR(200),
    phone VARCHAR(20),
    email VARCHAR(100),
    rating INTEGER CHECK (rating >= 0),
    discount INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Создание таблицы продукции
CREATE TABLE products_posohina (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    code VARCHAR(50) UNIQUE,
    cost DECIMAL(10, 2) CHECK (cost > 0)
);

-- Создание таблицы продаж
CREATE TABLE sales_posohina (
    id SERIAL PRIMARY KEY,
    partner_id INTEGER NOT NULL REFERENCES partners_posohina(id) ON DELETE CASCADE,
    product_id INTEGER NOT NULL REFERENCES products_posohina(id) ON DELETE RESTRICT,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    sale_date DATE NOT NULL DEFAULT CURRENT_DATE,
    total_sum DECIMAL(10, 2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

С целью автоматического обновления значения скидки при внесении изменений в информацию о продажах были разработаны специальные триггеры и соответствующие функции, код которых приведен в листинге 2.3.

Листинг 2.3 – Триггеры для автоматического расчета скидки

```
-- Функция расчета скидки
CREATE OR REPLACE FUNCTION calculate_discount_posohina(p_partner_id INTEGER)
RETURNS INTEGER AS $$
DECLARE
    total_quantity INTEGER;
    result_discount INTEGER;
BEGIN
    SELECT COALESCE(SUM(quantity), 0) INTO total_quantity
    FROM sales_posohina
    WHERE partner_id = p_partner_id;

    IF total_quantity < 10000 THEN
        result_discount := 0;
    ELSIF total_quantity < 50000 THEN
        result_discount := 5;
    ELSIF total_quantity < 300000 THEN
        result_discount := 10;
    ELSE
        result_discount := 15;
    END IF;

    RETURN result_discount;
END;
$$ LANGUAGE plpgsql;

-- Функция обновления скидки
CREATE OR REPLACE FUNCTION refresh_partner_discount_posohina(p_partner_id INTEGER)
RETURNS VOID AS $$
BEGIN
    UPDATE partners_posohina
    SET discount = calculate_discount_posohina(p_partner_id)
    WHERE id = p_partner_id;
END;
$$ LANGUAGE plpgsql;
```

После создания таблиц выполняется наполнение базы данных тестовыми значениями (листинг 2.4).

Листинг 2.4 – Наполнение базы тестовыми данными

```
-- Начальные данные
INSERT INTO partner_types_posohina (name) VALUES
    ('ООО'), ('ЗАО'), ('ИП'), ('АО'), ('ПАО');

INSERT INTO products_posohina (name, code, cost) VALUES
    ('Товар А-1', 'COD001', 1200.00),
    ('Товар Б-2', 'COD002', 2800.00),
    ('Товар В-3', 'COD003', 650.00),
    ('Товар Г-4', 'COD004', 3500.00),
    ('Товар Д-5', 'COD005', 1800.00);

-- Тестовые партнеры
INSERT INTO partners_posohina (type_id, name, director_fullname, phone, email, rating)
VALUES
    (1, 'Вектор', 'Смирнов Алексей Петрович', '+7(911)222-33-44', 'vector@mail.ru', 5),
    (2, 'Альянс', 'Козлова Елена Викторовна', '+7(922)333-44-55', 'alyans@yandex.ru', 4),
    (3, 'Соколов ИП', 'Соколов Дмитрий Игоревич', '+7(933)444-55-66', 'sokolov@bk.ru', 3);

-- Тестовые продажи
INSERT INTO sales_posohina (partner_id, product_id, quantity, sale_date)
VALUES
    (1, 1, 6000, '2024-02-10'),
    (1, 2, 4000, '2024-03-15'),
    (1, 3, 7000, '2024-04-20'),
    (2, 4, 25000, '2024-02-05'),
    (2, 5, 18000, '2024-03-18'),
    (3, 1, 2000, '2024-04-12'),
    (3, 3, 3000, '2024-05-25');
```

По окончании подготовительных этапов и формирования всех требуемых элементов базы данных было выполнено заполнение таблиц. Для этого использовалась учетная запись, созданная ранее. Построение структуры таблиц производилось строго по схеме, изображенной на рисунке 2.1.

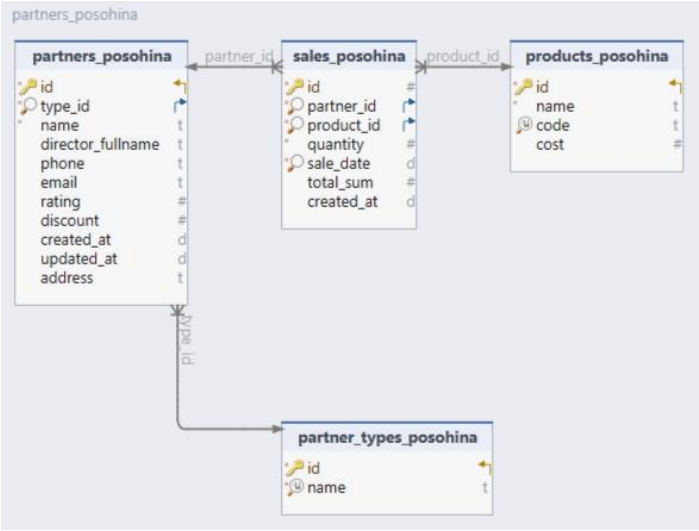

Рисунок 2.1 – Схема базы данных

В процессе создания таблиц были задействованы следующие форматы данных:

- SERIAL — числовой тип с автозаполнением, использовавшийся для формирования первичных ключей;
- INT — обычный целочисленный тип, применявшийся для хранения чисел и внешних ключей;
- VARCHAR(n) — строковый формат с фиксированной максимальной длиной;
- DATE — тип данных для хранения дат.

Для внешних ключей были установлены каскадные режимы обновления и удаления, что позволяет автоматически синхронизировать связанные записи при изменении или удалении родительских данных.

К полям, содержащим рейтинг партнера и количество реализованной продукции, добавлено условие, запрещающее ввод отрицательных значений (требование «больше или равно нулю»).

Для проверки корректности структуры в таблицы были внесены тестовые данные: перечень категорий партнеров и список товаров.

По завершении формирования базы данных и ее заполнения тестовыми записями был осуществлен переход к следующему этапу — разработке библиотек, обеспечивающих взаимодействие с базой данных на уровне программного кода.

Глава 3. Создание программных библиотек

В рамках выполнения работы были созданы три основных компонента для взаимодействия с базой данных:

- библиотека с моделями, отражающими структуру таблиц;
- библиотека с контекстом базы данных, отвечающим за соединения и выполнение запросов;
- библиотека, реализующая базовые операции с данными: чтение, добавление, изменение и удаление записей.

Для автоматической генерации моделей на основе существующей структуры базы данных был использован инструмент из пакета «Microsoft.EntityFrameworkCore.Tools». С помощью команды «Scaffold-DbContext» (представленной в листинге 3.1) были созданы классы-сущности на C#, соответствующие таблицам базы данных, а также сформирован контекст, через который осуществляется взаимодействие с этими таблицами. В дальнейшем контекст был выделен в отдельную библиотеку для обеспечения модульности архитектуры.

Листинг 3.1 – Команда для создания классов-сущностей по структуре базы данных

Для выполнения операции автоматического создания классов необходимо указать ряд параметров:

- строка подключения к базе данных, включающая адрес сервера, номер порта, название базы и данные для авторизации;
- указание провайдера, совместимого с используемой системой управления базами данных;
- пути к проектам, где уже установлен нужный пакет и куда будут помещены сгенерированные файлы;
- название схемы в базе данных, из которой будет производиться импорт;
- имя класса контекста, через который будет осуществляться доступ к данным.

После завершения работы команды и размещения сгенерированных файлов в соответствующих проектах была получена структура папок и файлов, представленная на рисунке 3.1.

```
Ссылок: 0
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseNpgsql("Host=localhost;Database=posohina;Username=app3;Password=123456789;Encoding=UTF8;Search Path=business");
    optionsBuilder.EnableSensitiveDataLogging();
    optionsBuilder.EnableDetailedErrors();
}
```

Для выполнения операции автоматического создания классов необходимо указать ряд параметров:

- строка подключения к базе данных, включающая адрес сервера, номер порта, название базы и данные для авторизации;
- указание провайдера, совместимого с используемой системой управления базами данных;
- пути к проектам, где уже установлен нужный пакет и куда будут помещены сгенерированные файлы;
- название схемы в базе данных, из которой будет производиться импорт;
- имя класса контекста, через который будет осуществляться доступ к данным.

После завершения работы команды и размещения сгенерированных файлов в соответствующих проектах была получена структура папок и файлов, представленная на рисунке 3.1.

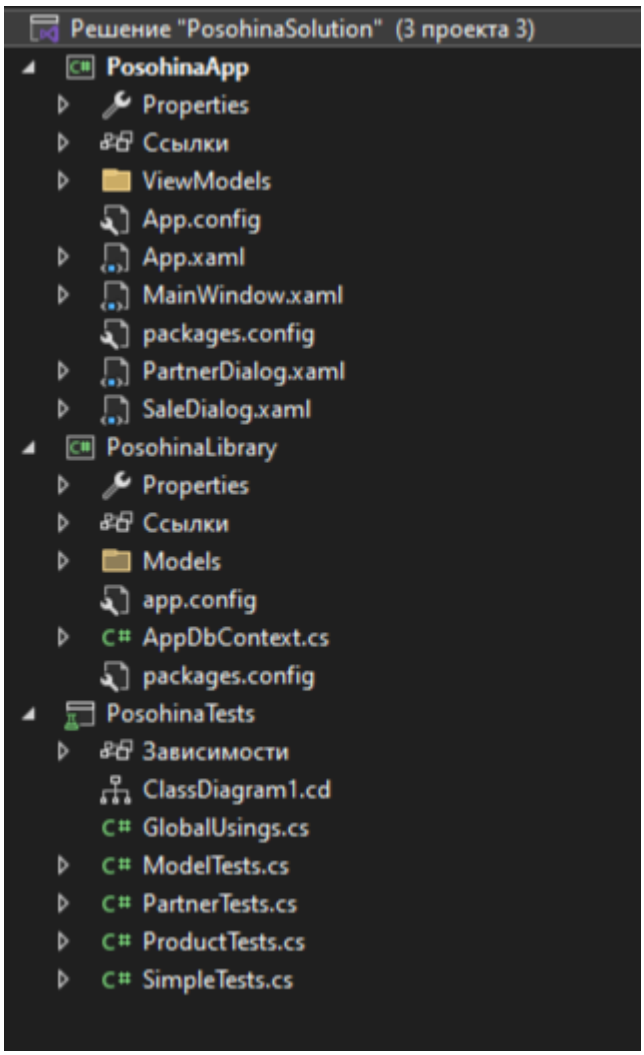

Рисунок 3.1 – Организация файлов и каталогов в проектах с моделями данных и контекстом

Графическое отображение состава автоматически созданных классов-моделей и класса контекста базы данных представлено на рисунках 3.2 и 3.3. На этих изображениях показаны наборы свойств для каждой модели, а также методы, предоставляемые контекстом для работы с таблицами.

```
PosohinaLibrary                                   PosohinaLibrary.AppDbContext                    OnConf
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
using PosohinaLibrary.Models;
using System;

namespace PosohinaLibrary
{
    Ссылок: 6
    public class AppDbContext : DbContext
    {
        Ссылок: 1
        public DbSet<PartnerType> PartnerTypes { get; set; }
        Ссылок: 3
        public DbSet<Partner> Partners { get; set; }
        Ссылок: 1
        public DbSet<Product> Products { get; set; }
        Ссылок: 5
        public DbSet<Sale> Sales { get; set; }

        Ссылок: 0
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseNpgsql("Host=localhost;Database=posohina;Username=app3;Password=123456789;Encoding=UTF8;Search Path=business");
            optionsBuilder.EnableSensitiveDataLogging();
            optionsBuilder.EnableDetailedErrors();
        }

        Ссылок: 0
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.HasDefaultSchema("business");

            modelBuilder.Entity<PartnerType>().ToTable("partner_types_posohina");
            modelBuilder.Entity<Partner>().ToTable("partners_posohina");
            modelBuilder.Entity<Product>().ToTable("products_posohina");
            modelBuilder.Entity<Sale>().ToTable("sales_posohina");

            modelBuilder.Entity<PartnerType>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Id).HasColumnName("id");
                entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(100);
            });

            modelBuilder.Entity<Partner>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Id).HasColumnName("id");
                entity.Property(e => e.TypeId).HasColumnName("type_id");
                entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(200);
                entity.Property(e => e.DirectorFullname).HasColumnName("director_fullname").HasMaxLength(200);
                entity.Property(e => e.Phone).HasColumnName("phone").HasMaxLength(20);
                entity.Property(e => e.Email).HasColumnName("email").HasMaxLength(100);
                entity.Property(e => e.Rating).HasColumnName("rating");
                entity.Property(e => e.Discount).HasColumnName("discount");
                entity.Property(e => e.CreatedAt).HasColumnName("created_at");
                entity.Property(e => e.UpdatedAt).HasColumnName("updated_at");

                entity.HasOne(e => e.PartnerType)
```

Рисунок 3.2 – Частичное содержание контекста базы данных

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace PosohinaLibrary.Models
{
    [Table("partners_posohina", Schema = "business")]

    public class Partner
    {
        [Key]
        [Column("id")]

        public int Id { get; set; }

        [Column("type_id")]

        public int TypeId { get; set; }

        [Required]
        [Column("name")]
        [MaxLength(200)]

        public string Name { get; set; }

        [Column("director_fullname")]
        [MaxLength(200)]

        public string DirectorFullname { get; set; }

        [Column("phone")]
        [MaxLength(20)]
        Ссылок: 4
        public string Phone { get; set; }

        [Column("email")]
        [MaxLength(100)]
        Ссылок: 3
        public string Email { get; set; }


        [Column("address")]
        Ссылок: 3
        public string Address { get; set; }


        [Column("rating")]
        Ссылок: 8
        public int? Rating { get; set; }

        [Column("discount")]
        Ссылок: 6
        public int Discount { get; set; }

        [Column("created_at")]
        Ссылок: 4
        public DateTime CreatedAt { get; set; }

        [Column("updated_at")]
        Ссылок: 5
        public DateTime UpdatedAt { get; set; }
```

Рисунок 3.3 – Содержание модели партнера

Из приведенных иллюстраций видно, что сгенерированные классы-сущности включают только навигационные поля для связи с другими объектами. Параметры соединения с базой данных (адрес сервера, имя пользователя, пароль), а также настройки таблиц, столбцов, ограничений и ключей сосредоточены в методах контекста.

Для организации работы с базой данных используются следующие элементы Entity Framework Core:

- DbContext — основной класс, через который устанавливается соединение и выполняются запросы;
- DbSet<TEntity> — набор, отображающий содержимое таблицы и позволяющий работать с соответствующими объектами;
- DbContextOptionsBuilder — средство для настройки параметров подключения;
- ModelBuilder — набор методов, задающих соответствие между классами и таблицами базы данных.

Главным классом для доступа к данным является AppDbContext, наследуемый от DbContext. В нем каждая таблица представлена свойством типа DbSet, параметризованным соответствующей моделью.

Далее разрабатываются сервисные классы, которые содержат логику работы с конкретными сущностями. Для каждого типа данных создается отдельный сервис, реализующий стандартные операции: получение списка, добавление, изменение и удаление записей. К каждому сервису также создается интерфейс, что позволяет снизить связанность компонентов. Примерное устройство интерфейса и реализующего его сервиса показано на рисунках 3.4 и 3.5.

```
using PosohinaLibrary.Models;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace PosohinaApp.ViewModels
{

    public class PartnerViewModel : INotifyPropertyChanged
    {
        private Partner _partner;


        public PartnerViewModel(Partner partner)
        {
            _partner = partner;
        }

        Ссылок 7
        public int Id => _partner.Id;
        Ссылок 2
        public string DisplayName => $"{_partner.PartnerType?.Name} / {_partner.Name}";
        Ссылок 1
        public string Director => _partner.DirectorFullname ?? "Не указан";
        Ссылок 1
        public string Phone => _partner.Phone ?? "Не указан";
        Ссылок 0
        public int? Rating => _partner.Rating;
        Ссылок 1
        public int Discount => _partner.Discount;


        Ссылок 0
        public string Address => _partner.Address ?? "Не указан";


        Ссылок 2
        public Partner Partner => _partner;

        public event PropertyChangedEventHandler PropertyChanged;
        Ссылок 0
        protected void OnPropertyChanged([CallerMemberName] string name = null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
        }
    }
}
```

Рисунок 3.4 – Структура интерфейса для взаимодействия с данными партнеров

```
using PosohinaLibrary.Models;
using System;

namespace PosohinaApp.ViewModels
{

    public class SaleViewModel
    {
        private Sale _sale;

        Ссылок 1
        public SaleViewModel(Sale sale)
        {
            _sale = sale;
        }

        Ссылок 2
        public int Id => _sale.Id;
        Ссылок 0
        public string ProductName => _sale.Product?.Name ?? "Неизвестно";
        Ссылок 0
        public int Quantity => _sale.Quantity;
        Ссылок 0
        public DateTime SaleDate => _sale.SaleDate;
        Ссылок 0
        public decimal TotalSum => _sale.TotalSum;
    }
}
```

Рисунок 3.5 – Пример части программного сервиса для работы с партнерами

В сервисные методы, отвечающие за добавление, изменение и удаление данных, передаются следующие сведения:

- при создании новой записи — объект, содержащий все поля, кроме первичного ключа, который формируется автоматически;
- при обновлении записи — идентификатор изменяемого объекта и новые значения для полей, подлежащих изменению;
- при удалении записи — идентификатор объекта, который требуется удалить.

Методы, выполняющие получение данных, в большинстве случаев не нуждаются в передаче параметров. Исключение составляет обращение к истории продаж: для вывода списка реализованной продукции конкретного партнера необходимо указать его идентификатор.

Перед выполнением любой операции с данными в сервисах производится проверка переданных значений на соответствие ограничениям, заданным структурой базы данных. Если обнаружены некорректные данные, вызывается исключение, которое прерывает выполнение операции и передает управление обратно вызывающему коду. Сформированное сообщение об ошибке содержит пояснение причины сбоя, что позволяет затем уведомить пользователя. В большинстве случаев проверка реализуется с помощью условных операторов. Для проверки данных партнера дополнительно используются регулярные выражения, контролирующие корректность телефонного номера и адреса электронной почты.

Для упрощения сигнатур методов и повышения удобства использования в некоторых случаях применяются вспомогательные объекты (DTO), объединяющие несколько связанных параметров. Это позволяет передавать все необходимые данные в виде одного объекта, сокращая количество аргументов методов.

Окончательная структура разработанных библиотек и связи между их компонентами показаны на диаграмме классов, представленной на рисунке

3.6.

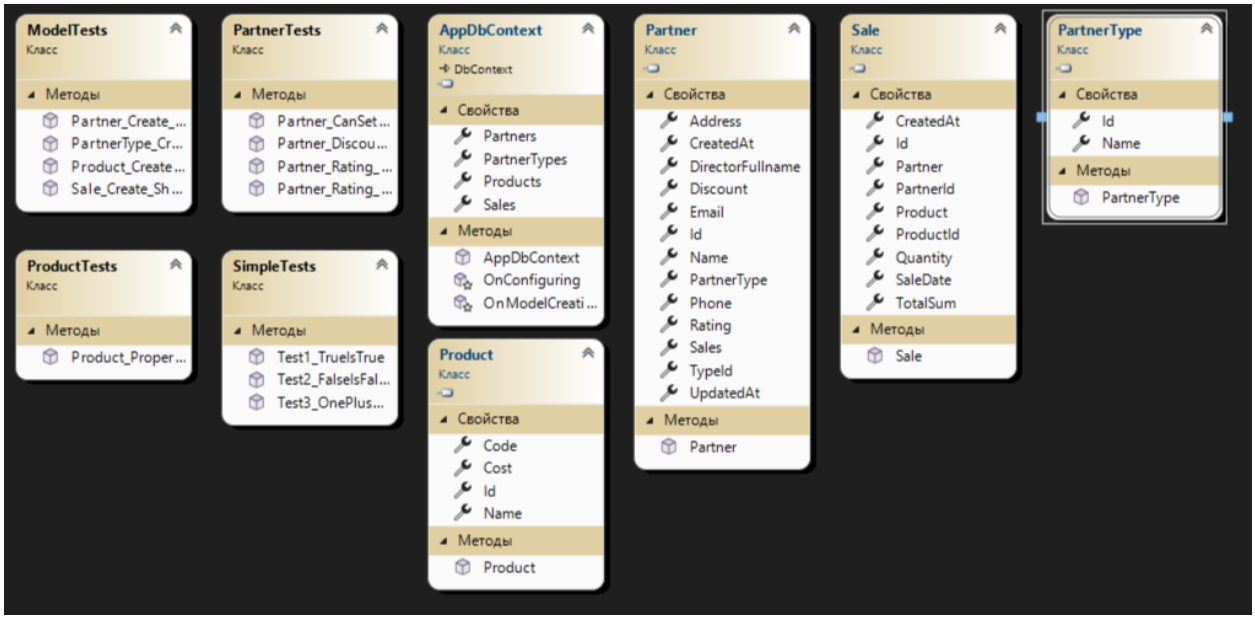

Рисунок 3.6 – Диаграмма классов

Глава 4. Тестирование библиотеки

4.1 Проверка работоспособности компонентов с помощью модульных тестов

Для проверки правильности работы ключевых компонентов разработанной системы был создан набор автоматизированных тестов с использованием фреймворка xUnit. Проверке подвергались модели данных, связующие классы ViewModel и алгоритм расчета индивидуальной скидки для партнера.

Проект с тестами реализован как независимая часть решения, не имеющая зависимостей от других компонентов приложения. Все проверки выполняются в изоляции, без подключения к реальной базе данных — тестируемые объекты создаются и обрабатываются непосредственно в оперативной памяти, что позволяет достичь высокой скорости выполнения и воспроизводимости результатов.

При тестировании моделей данных (результаты показаны на рисунке 4.1) проверялась корректность сохранения и возврата значений, присвоенных свойствам объектов. Для каждой модели выполнялась стандартная последовательность действий: создание экземпляра, заполнение полей тестовыми данными, чтение значений и их сравнение с ожидаемыми.

```
using Xunit;
using PosohinaLibrary.Models;

namespace PosohinaTests
{
    Ссылок: 0
    public class PartnerTests
    {
        [Fact]
        Ссылок: 0
        public void Partner_Discount_DefaultIsZero()
        {
            // Arrange
            var partner = new Partner();

            // Act
            var discount = partner.Discount;

            // Assert
            Assert.Equal(0, discount);
        }

        [Fact]
        Ссылок: 0
        public void Partner_CanSetDiscount()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Discount = 15;

            // Assert
            Assert.Equal(15, partner.Discount);
        }

        [Fact]
        Ссылок: 0
        public void Partner_Rating_CanBeNull()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Rating = null;

            // Assert
            Assert.Null(partner.Rating);
        }

        [Fact]
        Ссылок: 0
        public void Partner_Rating_CanBeSet()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Rating = 5;
```

Рисунок 4.1 – Тест модели партнера

При выполнении проверок, результаты которых представлены на рисунке 4.2, оценивалась корректность подготовки данных к отображению в интерфейсе, а также правильность обработки случаев, когда значения свойств моделей отсутствуют (равны null). Основное внимание уделялось поведению компонентов при отсутствии данных и корректному формированию строк для вывода на экран.

```
using Xunit;
using PosohinaLibrary.Models;
using System;

namespace PosohinaTests
{

    public class ModelTests
    {
        [Fact]

        public void Partner_Create_ShouldWork()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Id = 1;
            partner.Name = "Тестовый партнер";

            // Assert
            Assert.Equal(1, partner.Id);
            Assert.Equal("Тестовый партнер", partner.Name);
        }

        [Fact]

        public void Product_Create_ShouldWork()
        {
            // Arrange
            var product = new Product();

            // Act
            product.Id = 5;
            product.Name = "Тестовый продукт";
            product.Cost = 1000m;

            // Assert
            Assert.Equal(5, product.Id);
            Assert.Equal("Тестовый продукт", product.Name);
            Assert.Equal(1000m, product.Cost);
        }

        [Fact]

        public void Sale_Create_ShouldWork()
        {
            // Arrange
            var sale = new Sale();
            var now = DateTime.Now;

            // Act
            sale.Id = 10;
            sale.Quantity = 5;
            sale.TotalSum = 5000m;

            // Assert
            Assert.Equal(10, sale.Id);
            Assert.Equal(5, sale.Quantity);
            Assert.Equal(5000m, sale.TotalSum);
        }

        [Fact]

        public void PartnerType_Create_ShouldWork()
        {
            // Arrange
            var type = new PartnerType();

            // Act
            type.Id = 3;
            type.Name = "ООО";
```

Рисунок 4.2 – Тест модел

4.2 Результаты тестирования

Все подготовленные автоматические тесты были успешно выполнены, ошибок не зафиксировано. Каждый компонент системы показал работу, соответствующую ожидаемой и требованиям технического задания. Итоговые данные о прохождении тестов приведены на рисунке 4.3.

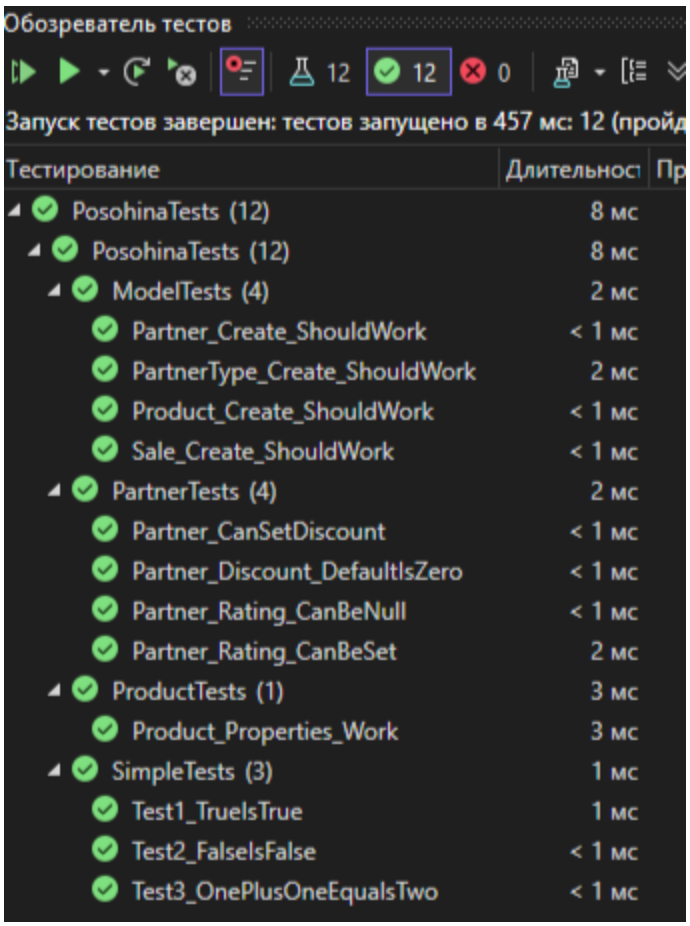

Рисунок 4.3 – Результаты выполнения тестов

Выполнение всех тестов без ошибок свидетельствует о правильной работе ключевых модулей системы и их пригодности для включения в состав клиентского приложения.

Глава 5. Создание клиентского приложения

5.1 Разработка пользовательского интерфейса

Для реализации пользовательского интерфейса была выбрана технология Windows Presentation Foundation (WPF), которая позволяет создавать современные оконные приложения с широкими возможностями настройки внешнего вида.

В ходе проектирования были выделены три ключевые формы, через которые осуществляется взаимодействие с системой:

- Главное окно — основной экран приложения, на котором одновременно представлены список контрагентов и информация о продажах текущего выбранного партнера;
- Форма для работы с данными партнера — окно, через которое выполняется добавление новых записей и редактирование существующих сведений о партнерах;
- Форма регистрации продаж — окно, предназначенное для внесения и изменения данных о реализованной продукции.

Внешний вид главного окна показан на рисунке 5.1. Его рабочая область поделена на две части: слева отображается перечень всех партнеров, справа — сведения о продажах, относящихся к выбранному контрагенту. В верхней части окна размещено главное меню, обеспечивающее доступ ко всем функциям приложения.

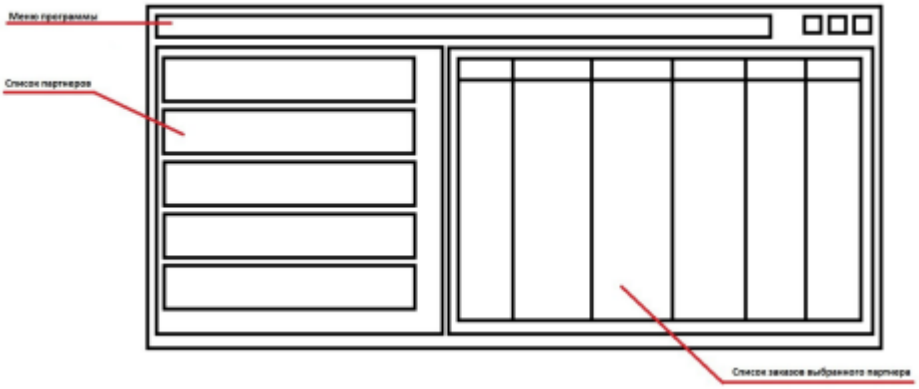

Рисунок 5.1 – Прототип главного окна

С целью улучшения пользовательского опыта в приложение были добавлены следующие элементы:

- контекстное меню — вызывается правой кнопкой мыши и содержит команды, применимые к текущему выбранному элементу;
- горячие клавиши — назначены для часто используемых действий: F5 обновляет данные в списках, F1 открывает справочную информацию, F2 позволяет редактировать выбранную запись, Delete удаляет выделенный объект;
- визуальные подсказки — цветовое выделение активных элементов и понятные сообщения об ошибках помогают пользователю лучше ориентироваться в приложении.

5.2 Построение окон приложения

Главное окно программы (показано на рисунке 5.2) создано с использованием стандартных компонентов Menu, ListBox и DataGrid. Список партнеров отображается через ListBox с применением специально разработанного шаблона элемента, в котором выводится вся ключевая информация о контрагенте, включая значение текущей скидки. История продаж представлена в виде таблицы DataGrid с фиксированным набором колонок для отображения требуемых данных.

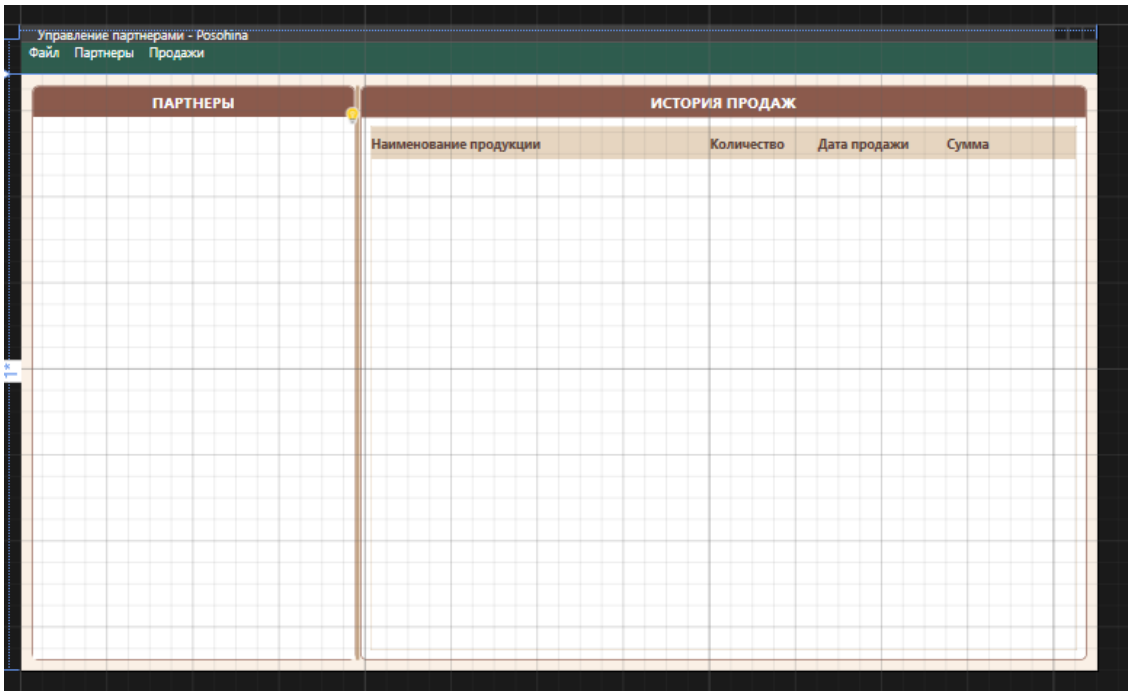

Рисунок 5.2 – Фрагмент разметки главного окна

Окно добавления и изменения данных партнера (рисунок 5.3) содержит поля, соответствующие всем обязательным сведениям о контрагенте. Поле рейтинга принимает только цифры, что предотвращает случайный ввод некорректных символов. Поле скидки недоступно для редактирования и предназначено лишь для отображения текущего значения, которое рассчитывается автоматически.

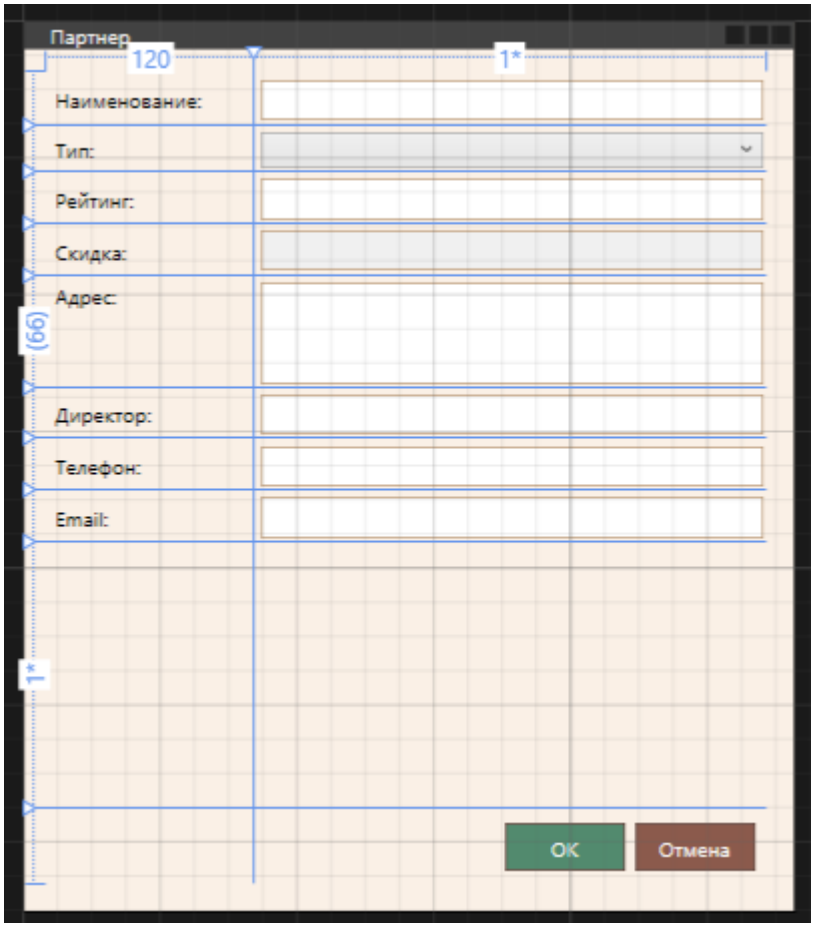

Рисунок 5.3 – Диалоговое окно партнера

Окно оформления продажи (рисунок 5.4) содержит выпадающий список для выбора товара, поле ввода количества с проверкой допустимых значений, элемент выбора даты и автоматически рассчитываемую сумму.

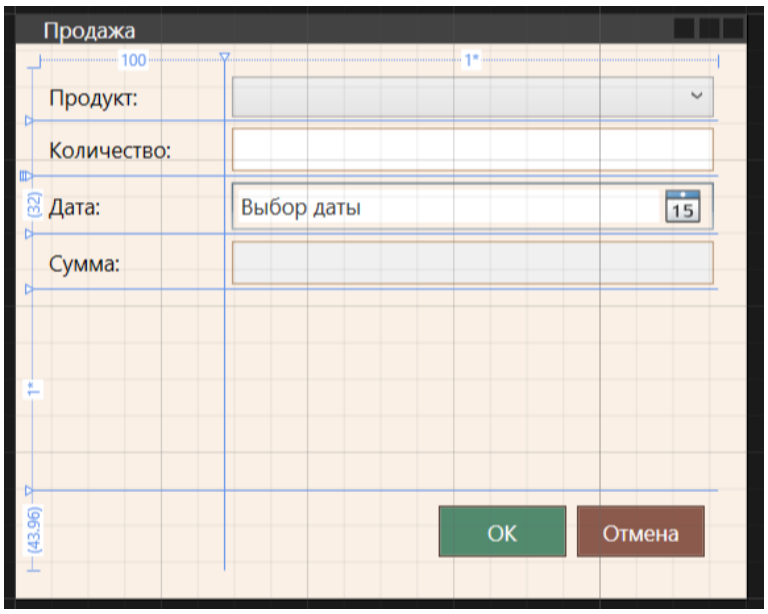

Рисунок 5.4 – Диалоговое окно продажи

Для обеспечения единообразного внешнего вида всех окон в приложении используются общие стили, заданные в ресурсах. Пример такой настройки приведен на рисунке 5.5.

```
<Application x:Class="PosohinaApp.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="MainWindow.xaml">
    <Application.Resources>
        <Style TargetType="Button">
            <Setter Property="Padding" Value="10,5"/>
            <Setter Property="FontSize" Value="12"/>
            <Setter Property="Background" Value="#8B5A4C"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="BorderBrush" Value="#5A3E35"/>
            <Style.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="#A87564"/>
                </Trigger>
            </Style.Triggers>
        </Style>

        <Style TargetType="TextBox">
            <Setter Property="FontSize" Value="12"/>
            <Setter Property="Padding" Value="5"/>
            <Setter Property="BorderBrush" Value="#C0A080"/>
            <Style.Triggers>
                <Trigger Property="IsFocused" Value="True">
                    <Setter Property="BorderBrush" Value="#D2694A"/>
                </Trigger>
            </Style.Triggers>
        </Style>

        <Style x:Key="DataGridHeaderStyle" TargetType="DataGridColumnHeader">
            <Setter Property="Background" Value="#E6D5C0"/>
            <Setter Property="Foreground" Value="#5A3E35"/>
            <Setter Property="FontWeight" Value="Bold"/>
            <Setter Property="Height" Value="30"/>
        </Style>

        <Style x:Key="DataGridCellStyle" TargetType="TextBlock">
            <Setter Property="Margin" Value="5,0,0,0"/>
            <Setter Property="VerticalAlignment" Value="Center"/>
        </Style>

        <Style x:Key="DataGridCellStyleCenter" TargetType="TextBlock">
            <Setter Property="HorizontalAlignment" Value="Center"/>
            <Setter Property="VerticalAlignment" Value="Center"/>
        </Style>

        <Style x:Key="DataGridCellStyleRight" TargetType="TextBlock">
            <Setter Property="HorizontalAlignment" Value="Right"/>
            <Setter Property="Margin" Value="0,0,5,0"/>
            <Setter Property="VerticalAlignment" Value="Center"/>
        </Style>
    </Application.Resources>
</Application>
```

Рисунок 5.5 – Глобальные стили приложения

В главном окне предусмотрена обработка нескольких событий: загрузка данных в списки при запуске приложения, вывод информации о продажах при выборе конкретного партнера, а также вызов дополнительных окон при выборе соответствующих пунктов меню. Код обработчиков показан на рисунке 5.6.

```
Ссылок: 8
private void LoadData()
{
    try
    {
        _context = new AppDbContext();

        var partners = _context.Partners
            .Include(p => p.PartnerType)
            .Include(p => p.Sales)
                .ThenInclude(s => s.Product)
            .ToList();

        _partners = partners.Select(p => new PartnerViewModel(p)).ToList();
        PartnersListBox.ItemsSource = _partners;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки: {ex.Message}", "Ошибка",
                        MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Рисунок 5.6 – Метод загрузки данных

Во вспомогательных окнах осуществляется контроль правильности заполнения полей (пример приведен на рисунке 5.7), а также перехватываются и обрабатываются сбои, возникающие в процессе сохранения информации.

```
Ссылок: 1
private void SaveMenuItem_Click(object sender, RoutedEventArgs e)
{
    try
    {
        _context.SaveChanges();
        MessageBox.Show("Данные сохранены", "Информация",
                        MessageBoxButton.OK, MessageBoxImage.Information);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
                        MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

Ссылок: 1
private void ExitMenuItem_Click(object sender, RoutedEventArgs e)
{
    Close();
}

Ссылок: 1
private void SaveAsTxtMenuItem_Click(object sender, RoutedEventArgs e)
{
    var dialog = new SaveFileDialog
    {
        Filter = "Текстовые файлы (*.txt)|*.txt",
        DefaultExt = "txt",
        FileName = $"partners_{DateTime.Now:yyyyMMdd_HHmmss}.txt"
    };

    if (dialog.ShowDialog() == true)
    {
        try
        {
            using (var writer = new StreamWriter(dialog.FileName, false, Encodi
            {
                writer.WriteLine("=== ПАРТНЕРЫ ===");
                writer.WriteLine($"Дата: {DateTime.Now:dd.MM.yyyy}");
                writer.WriteLine(new string('-', 50));

                foreach (var partner in _partners)
                {
                    writer.WriteLine($"Партнер: {partner.DisplayName}");
                    writer.WriteLine($"Директор: {partner.Director}");
                    writer.WriteLine($"Телефон: {partner.Phone}");
                    writer.WriteLine($"Скидка: {partner.Discount}%");
                    writer.WriteLine(new string('-', 50));
                }
            }

            MessageBox.Show("Файл сохранен", "Успех"
```

Рисунок 5.7 – Валидация и сохранение данных

Окончательный вариант оформления основного окна программы показан на рисунке 5.8.

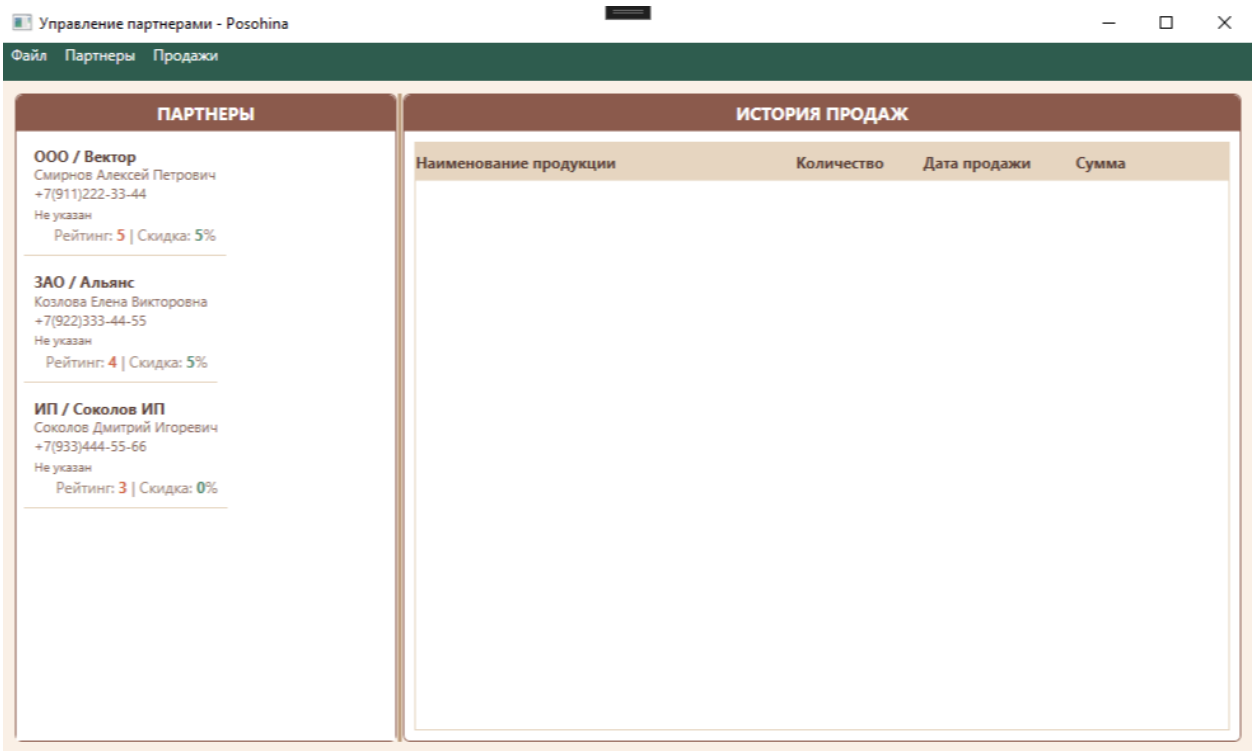

Рисунок 5.8 – Главное окно приложения

Заключение

В ходе производственной практики, проходившей с 9 по 17 марта 2026 года, были систематизированы теоретические знания и усовершенствованы практические умения в области разработки программного обеспечения.

Основные итоги работы:

- На основе анализа проектной и технической документации были определены требования к программным модулям, что позволило обеспечить их корректное взаимодействие в рамках общей архитектуры создаваемого ПО.
- Проведено моделирование процессов разработки, благодаря чему удалось структурировать этапы интеграции компонентов и установить логические связи между ними.
- Выполнена отладка программных элементов с использованием специализированных инструментов, что способствовало своевременному выявлению и устранению ошибок.
- Разработаны комплекты тестов и сценарии проверки, позволившие оценить надежность и корректность функционирования программного продукта.

Полученные практические навыки:

- интеграция отдельных модулей в единую работающую систему;
- выявление и устранение ошибок в коде с помощью отладочных средств;
- составление тестовых наборов и сценариев для проверки работоспособности ПО;
- анализ исходного кода на соответствие принятым стандартам оформления.

Приложения

ПРИЛОЖЕНИЕ 1

Описание предметной области

Производственная компания «Мастер пол» выпускает напольные покрытия. Компания занимается производством и реализует свою продукцию через партнеров, которые доставляют продукцию компании до конечных потребителей.

С целью оптимизации деятельности компании разрабатывается система, в которой заказчик выделил несколько подсистем:

- продукция;
- склад и материалы;
- производство;
- сотрудники;
- партнеры.

В данной области определены следующие основные составляющие:

1. Партнеры: тип, наименование компании, юридический адрес, ИНН, ФИО директора, контактные данные (телефон, email), логотип, рейтинг, места продаж, история реализации продукции для последующего расчета скидок.

Партнеры покупают продукцию со скидками, которые зависят от общего объема их продаж. Скидки мотивируют партнеров к большему объему продаж. Партнеры реализуют товары в розничных и оптовых магазинах, интернет-магазинах, другим компаниям по продаже товаров и оказанию услуг.

Взаиморасчеты производственной компании с партнерами осуществляются в рублях.

2. Менеджеры: поиск и регистрация партнеров, решение об изменении рейтинга партнера, ведение истории изменений рейтинга, прием

заявок от партнеров, формирование для них предложения на основании статистики истории продаж, полученной от аналитиков компании.

Продолжение приложения 1

3. Заявка: если партнер ранее производил закупку продукции и предоставил данные о ее реализации, то на основании этих данных и текущих объемах готовой продукции менеджер формирует предложение. Если работа ведется с новым партнером, то менеджер предоставляет каталог готовой продукции и принимает заявку. Затем менеджер должен указать стоимость и дату производства каждой указанной в заявке единицы продукции. После согласования списка продукции с количеством, стоимостью и сроком изготовления заявка считается созданной.

Партнер может отменить заявку до момента внесения предоплаты. Если предоплата не поступает в течение трех дней, то заявка отменяется автоматически, о чем обязательно уведомляется партнер. После поступления предоплаты продукция, указанная в заявке и отсутствующая на складе в нужном количестве, запускается в производство. По готовности всей продукции менеджер должен предложить организовать доставку. В момент получения продукции (путем доставки или самовывоза) партнер производит полную оплату. После проверки количества и качества продукции партнером заявка считается выполненной.

4. Сотрудники: ФИО, дата рождения, паспортные данные, банковские реквизиты, наличие семьи, состояние здоровья.

5. Кадры: допуск к работе с конкретным оборудованием. Мастер производства занимается технологией производства, контролем качества готовой продукции, заказом материалов у поставщиков.

6. Доступ: двери открываются при использовании карты сотрудника, система турникетов передает данные о перемещении сотрудников в приложение для формирования аналитиками статистики по сотрудникам.

7. Материалы (сырье): тип, наименование, поставщик, количество в

упаковке, единица измерения, описание, изображение, стоимость, количество на складе, минимальное допустимое количество, история изменений количества

Продолжение приложения 1

материалов на складе.

8. Склад: регистрация поступлений материалов, резервирование материалов для производства, отпуск и списание материалов со склада в производство, поступление и выбытие готовой продукции, получение информации о текущих остатках и движении товарно-материальных ценностей по складу.

9. Поставщики: тип, наименование, ИНН, история поставок материалов. Менеджеры контролируют список поставщиков, аналитики формируют информацию о качестве поставляемых материалов для принятия решения о прекращении работы с определенными компаниями.

10. Продукция: артикул, тип, наименование, описание, изображение, минимальная стоимость для партнера, размер упаковки (длина, ширина, высота), вес без упаковки, вес с упаковкой, сертификат качества (скан документа), номер стандарта, история изменений минимальной стоимости для партнера, время изготовления, себестоимость, номер цеха, количество человек на производстве, необходимые материалы для производства.